（03）で 、 と した。で 、そ だけで 「 き」 せ 。「 した す 」「 した ー ー 」「 じて ー す 」――こうした す が と、そ であ です。

こ で 、 ずJavaScript／TypeScript し、そ うえで Login.tsx ・ KosuList.tsx ・ axios.ts き す。とく 「 したとき が き か（handleSubmit）」と「 ー ーと す（async / await）」 、たとえ えて し す。

こ いですが、 してい す。 で「map ってどう く だっけ」「async ？」と った 、ここ って いてください。

- ― かす 。React
- ― ・ （const / let）・ ー ・ （===）・ ー ・if ・for ・ ・ ー
- ― map ・ filter 、 、 、 、 （?.）
- ― ・ ・interface ・ ・ ・union 。「 ぜ が か」
- ― Promise ・ async / await 、 でたとえて
- ― たいで ー い すしく
- ― ー ーと
- ―
-

0. ：JavaScriptとTypeScriptって　？
1. き
2. ― け　（const と let）
3. ー　―
4. ― と　（===　）
5. ー　―
6. ― if / else /　/ switch
7. し ― for と　ー
8. ― けて
9. ー　―　い　き
10. ―　て　つ
11. ― map と filter
12. ―　き
13. ・　・
14. TypeScript ―　という
15. ― Promise と async / await
16. import / export ―　たぐ
17. ー　：axios.ts　8
18. ー　：Login.tsx　handleSubmit
19. ー　：KosuList.tsx　と　とmap
20. ー
21.
22. こ　と

03　で、　3つ　でできてい　と　した。　う　おさ　いし　す。

	・	（ ・ ）
	た	・

HTMLとCSSだけだと、 「 」 う していい す。 くこと できて 、「 した かが き 」 う できな せ 。そ 「 した ～す 」 す が で す。

> と と「Web で かすた 」として た 。 して と き す。 で だけで く、 ー ー（Node.js） ど々 で き すが、 で 「 で かす」た す。⚠️ が てい すが です。「 」と「 」く い い す。 し い う 。

`.tsx` `.ts` という です。こ です。

> JavaScript 。 して 。Microsoftが した。JavaScript できこと でき、さ 「こ 」「こ 」という （ ） け す。こ 、 いてい で い づけ す（§14で しく）。
>
> TypeScript 、 。 TypeScript できな いか です。こ 、 で で で、あ たが す あ せ 。

> `.ts` … TypeScript（ だけ）。 で `axios.ts` がこ 。
> `.tsx` … TypeScript （HTML う 、`03` §3 ） け 。React こち 。`Login.tsx`・`KosuList.tsx` がこ 。

§1〜§16　　です。　　さ　（「※　　」と　）で
す。§17〜§19　　1　ずつ　　す。　　、　　ー　で　か
　　です。

　　「　　」と　、§17　　ー　で「あ、こ　さっき　　だ」と　づけ
　　です。

ー　　く「　　き」です。　　ー　　し、　が　た　だけ　　です。

> 　　き　　し　い「　　け
> 」　く　　し　す。

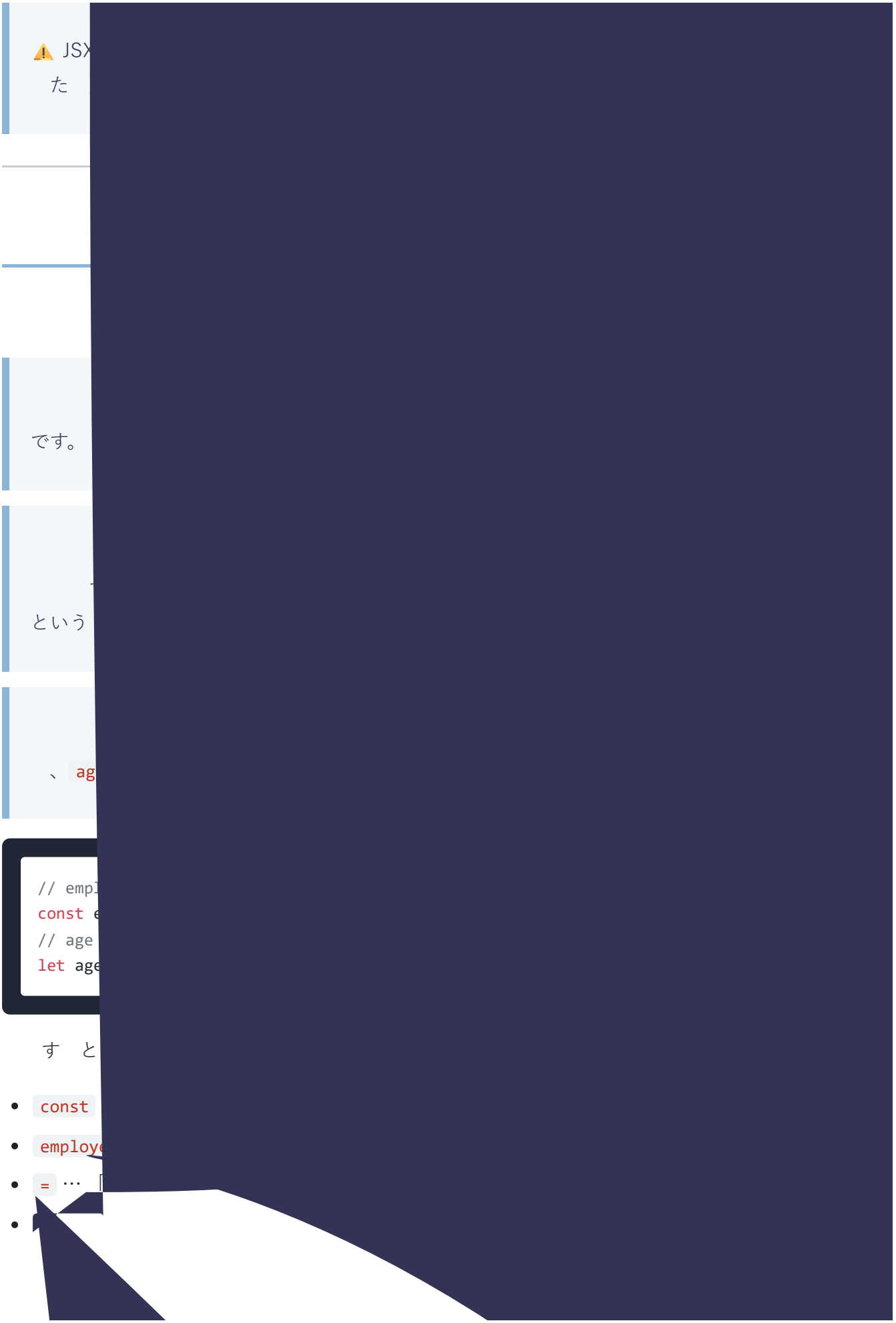
```
// こ　　。// か　　が　さ　　※

//　　　　こ　き
co　　

　　。
　こ　がす　　さ　。
　い　　き　い　き　う。
*/
```

- `//` … そ　、`//` か　だけが　（　）。　。
- `/* ... */` …　だ　す　てが　（　）。

> 　　ー　「　す　か（What）」　　か　すが、「　ぜそうす　か（Why）」
> いた　しか　か　せ　。　　た　、「　ぜこ　が　か」
> 　で　す　が　　です。



JavaScriptで　　2つ。　　です。

`const`	（constant）。　　た　え　い	❌　え　　い
`let`	。　で　　え	✅　え

> 　　`const`　　え　うとす　と　ー、`let`　　え　こと　し　す。

```
//　　。　　い　　const　※
const pi = 3.14;
// ❌　　const　　え　　い
pi = 3.15;

//　　。　　す　　let
let count = 0;
// ✅ OK。let　　え
count = 1;
// ✅ OK。　　count(1)　1　し　count　　す → 2
count = count + 1;
```

> 　　「え　つ　　い」`const`　しておくと、うっかき　えてし　う　　。　「ず`const`で　き、どうして　え　　があ　ときだけ`let`」という　です。　　ー　て　、`const`が　　です。

> ⚠️　`var`（ー）という　ー　ー　あ　したが、　　しい　があ　た　、　で　　。　　てき　せ　。`const`と`let`だけ　え　OKです。

- ・ ・`_`・`$` が え 。ただし （`1name` 、`name1` OK)。
- と さ （`age` と `Age`)。
- が か け （`x` `employeeName` うが)。
- つ ぐとき （`employeeName` う 2 ）が 。

> `searchDayRef` う 、 (camel) う が っ か 。 こ です。`employeeName`、

「 」 、いくつか があ す。 さえ す。

> ・ ・ ・null・undefined という、JavaScript って す。

```
//         string 。"" か ''         ※
const name = "         ";
//       number 。             じ number
const age = 42;
//         boolean 。true      か false        2
const isAdmin = true;
// null         。「          」     す
const nothing = null;
// undefined                    。「             い  い」
let notYet;
```


	string	。 で	`" "`, `'abc'`
	number	。 し	`42`, `3.14`, `-5`
	boolean	「 い/いいえ」 2	`true`, `false`
null	null	「 っ 」	`null`
undefined	undefined	「 だ ってい い」	`undefined`

> 。`"123"`
> （ できい）で、`123` （ できい）です。こ い でとて す（§18 で `Number()` が てき す）。

> 「
> か？」「 か？」 う Yes/No す い す。 `interface Kosu`
> `judgement: boolean`（ OK/NG）がこ です（§19）。

> どち 「 が い」ですが、 が い す。
>
> - `null` …「ここ です 」と た 。
> - `undefined` …「 ったが、 だ てい い」という 。 で「 ー が だ てい い＝`null`」 う い け す。 `useState<string | null>(null)`（§19） 「 ー し＝null」という い です。

あ が か `typeof` い す。

```
// "string"  ※
typeof "    "
// "number"
typeof 42
// "boolean"
typeof true
```

と 、`+` `===` う 「 ・ 」で す。

```
// 13 し ※
const a = 10 + 3;
// 7 き
const b = 10 - 3;
// 30 け 。× く *
const c = 10 * 3;
// 3.333... 。÷ く /
const d = 10 / 3;
// 1 。10 3 。「 」 いう
const e = 10 % 3;
```

「 か か」(`x % 2` が 0) ど い す。`03` CSS `nth-child(even)` があ したが、JS で `%` で くし す。

`+` 、 どうしだと「つ げ 」 す。

```
// "こ 、 さ " ※
const greeting = "こ 、" + " さ ";
// " 123 す" が さ が
const message = " " + 123 + " す";
```

⚠️ `+` 「 どうし し 、 が じ と 」という があ す。`"1" + 2` **`3` で く `"12"`** ( ) す。こ が ことがあ

> で、§5 ー うが です。

2つ て、`true` か `false` し す。

```
// true 5 3 きい ※
5 > 3
// false
5 < 3
// true
5 >= 5
// false
5 <= 4
// true しい
5 === 5
// true しく い
5 !== 3
```

> JavaScript 「 しい」 が2 あ す。
>
> - `==`（ ー 2つ）… 。 が って 「だいたい じ」 true してし う。`"5" == 5` （ 5と 5 じ い）。
> - `===`（ ー 3つ）… 。 で し いと true し い。`"5" === 5` （ と ）。
>
> `==` で た 、 JavaScriptで が で す。 ー `===` だけ ってい す。`Login.tsx` `data.status === "success"`（§18）、`KosuList.tsx` `value === ""`、`err.response?.status === 401`（§19） ど、す て `===` です。

> ⚠️
>
> - `=`（1つ）… （ ）。`x = 5` 「x 5 」。
> - `===`（3つ）… （ しいか ）。`x === 5` 「x 5と しいか？」。 `if (x = 5)` と いてし いがちですが、こ 「 」 つ が「 」 って

> す。 ず `===` 。

せ す。

```
// true && 「か AND 」。 true true ※
true && true
// false
true && false
// true || 「 OR 」。 かtrue true
true || false
// false ! 「 い NOT 」。 させ
!true
```

`&&`	AND	と true ときtrue	「かつ」
`||`	OR	どち かがtrue true	「 た 」
`!`	NOT	true/false	「〜で い」

> `{errorMessage && (<div>...</div>)}` （ `03` §3 で た ）、「 `errorMessage` があ ときだけ 」という です。React い ですが、 っこ `&&` です（ `05` で しく）。

> `response.data.results || []` （§19 `KosuList.tsx` ）、「 `results` があ そ い、 け （ っ ） `[]` う」という「 」です。 で です。

、 き です。

が18　　「　」、そうで　け　「　　」と　し
す。

```
//　※
const age = 20;

//　し age が18　　( )
if (age >= 18) {
  //　こ　{ }
  console.log("　　");
//　う　け
} else {
  //　こ　　{ }
  console.log("　　");
}
```

す　と：

- `if (　) { ... }` …　が `true`　`{ }`　。
- `else { ... }` …　が `false`　とき　(　)。
- `( )`　、　`{ }`　く。

(`03` §14.3)　し　す。「こ　が　っ
てい　か」　す　た　、　(　)　ー　です。

3つ　したいとき　`else if`　さ　す。

```
//   ※
if (score >= 80) {
  console.log("  ");
//   がfalse  、こ  がtrue
} else if (score >= 60) {
  console.log("  ");
//       false
} else {
  console.log("  ");
}
```

`if/else` き です。 で し す。

```
//          ? true   き    : false   き
//   ※
const result = age >= 18 ? "    " : "      ";
// age >= 18 が true      "    "、false      "      " が result
```

- `  ? A : B` … がtrue A、false B す。
- `?` が 、`?` と `:` がtrue 、`:` がfalse 。

> - `Login.tsx`：`setNumber(value === "" ? "" : value)` →「 が 、そうで け そ 」 す （§18）。
> - `KosuList.tsx`：`{item.judgement ? "OK" : "NG"}` →「judgementがtrue "OK"、false "NG" 」（§19）。
> - `KosuList.tsx`：`mode ? "month" : "day"` →「month ー "month"、そうで け "day"」（§19）。

1つ が「1 ～、2 ～、3 ～」と か とき、`if` すいが `switch` です。

1〜6 じて、 す し す。
`formatTyoku`（§19で ー ） そ です。

```
// value ※
switch (value) {
  // value が 1 き
  case 1:
    //  "1 " し switch け
    return "1 ";
  // value が 2 き
  case 2:
    return "2 ";
  case 3:
    return "3 ";
  //  case い き
  default:
    //  す
    return "";
}
```

す と：

- `switch ( ) { ... }` … 、 `case` と す 。
- `case :` …「 がこ と した 」。 した case が さ 。
- `return ...` … して え （`return` が い `break` で け あ 。§19で ）。
- `default:` … ど `case` し かったとき け （`if` `else` ）。

「1つ 、たくさ と す 」 `switch` が し す。「 （ AND/OR）」 `if` が き す。 「 （ ）→ 」 `switch` でい す。

じ すしく です。

0か　4　で　　5　、　　ー　　し　す。

```
// i　0か　　、5　　、　1ず　　す　※
for (let i = 0; i < 5; i++) {
  // i　　0,1,2,3,4　5
  console.log(i);
}
```

`for`　　　

§11 `map`・`filter` という、 っと さ た うか です。 ー `for` てこず、`map` が です。ただし `for` え （ し） として で さえておきし う。

「 」 してください。お （ ） て すと、 ー（ ）が てき す。 せず、「お → ー が 」だけ え す。 じで、「 す→ が 」 、 せず で え す。

2つ け って し した す `add` という 、そ い す。

```
// add いう 。a b け ※
function add(a, b) {
  // a + b 「 す」
  return a + b;
}

// す。3 5 す 8 が 、result
const result = add(3, 5);
// 8
console.log(result);
```

す と：



```
//                          、                    す ※
<button onClick={() => handleSearch(true)}>      </button>
```

こ KosuList.tsx ー （§19）そ です。「onClick（ さ たとき）、handleSearch(true) ー す」という です。

onClick={handleSearch(true)} と くと、 さ てし い す。onClick={() => handleSearch(true)} と「 で 」ことで、「 さ た て す 」という ができ す。こ い 05 で し てき す。

- Login.tsx：const handleSubmit = async (event) => { ... }（ ）
- Login.tsx：onChange={(e) => { ... }}（ が ったとき ）
- KosuList.tsx：const formatTyoku = (value) => { ... }、const getDayOfWeek = (dateStr) => { ... }
- KosuList.tsx：data.map((item) => (...))（ map）す て ー です。=> た「こ だ」と う し し う。

「 」。1つ （ ）、（ ）が で ってい す。「3 」と え せ す。

7 た 、 して し
す。 `getDayOfWeek`（§19） `days` と じ です。

```
// 7 ※
const days = ["  ", "  ", "  ", "  ", "  ", "  ", "  "];

// "  " 。 0か
console.log(days[0]);
// "  "
console.log(days[1]);
// "  "
console.log(days[6]);
// 7 。length 「 さ」
console.log(days.length);
```

す と：

- `[ ... ]` … で 、 で って 。
- `[ ]` … （ ）で す。
- `` … （ ）。

（ ） す。
が `[0]`、2 が `[1]`、…、 が `[length - 1]`。 ー
で、 が つ ずく です。「1 =0 」と え し う。
`getDayOfWeek` で `days[date.getDay()]` としてい す。`getDay()` 0・ 1…
6で す で、`days` と し す（§19で ）。

```
//   ※
const list = ["a", "b"];
//        "c"        → ["a","b","c"]
list.push("c");
// 0       "X"        → ["X","b","c"]
list[0] = "X";
```

> ⚠️ const で った で 、 。const が す 「 ご と え こと」だけ。list = [...]（ ） ですが、list.push()（ す） OKです。

うええで （ ）です。Reactで for こ い す。 し す。

> 名前 days.length length う 、そ 「くっついた 」 ことです。

> 2 した、 しい す。

```
//   ※
const numbers = [1, 2, 3];
//        n    n*2
const doubled = numbers.map((n) => n * 2);
// [2, 4, 6]       numbers          い
console.log(doubled);
```

す と：

- `.map(( ) => )` … して、 し、 す。
- `(n) => n * 2` …「1 `n` け 、`n * 2` す 」 ー 。

> 「 ー」。 が1つずつ ー てきて、`map` ( ) 、 さ た で しい っていく。 ( ) そ す。

> Reactで 「 ー 、 (・ ー ど) す 」 `map` い す。 `KosuList.tsx` で 、 ー ーか た ー `data` 、 `<tr>` してい す（§19）：
>
> ```
> {data.map((item) => (
>   <tr key={item.id}> ... </tr>   //  item 、1  <tr>
> ))}
> ```
>
> 「100 ー があ 、100 `<tr>` ができて、 100 さ 」という で す。

> か 、 だけ した しい す。

```
//  ※
const numbers = [1, 2, 3, 4, 5, 6];
// 「2  が0」  け す
const evens = numbers.filter((n) => n % 2 === 0);
// [2, 4, 6]
console.log(evens);
```

す と：

- `.filter(( ) => )` … が `true` しい す。
- `(n) => n % 2 === 0` …「n 2で った が0（= ）か？」という 。true だけ 。

> 「 い（ こし)」。 い かけ、 う （ ） だけが 。

> - `map` … 、 す （3 →3 ）。
> - `filter` … 、 う だけ （6 → 3 ）。
>
> で 、 ー `|| []`（§4.4）で「 が け 」 してか `map` で す 、という が です。 `filter` で って い せ が、API （`filter: "true"` ー 、§19）として「 」 が し す。

```
//  ※
const numbers = [1, 2, 3, 4, 5, 6];
const result = numbers
  //  ず  け す → [2,4,6]
  .filter((n) => n % 2 === 0)
  //  10  → [20,40,60]
  .map((n) => n * 10);
```

`.` でつ げて「 でか 」 う できす（ ー とす)。

が「　で　」
し、　「　で　」し　す。

「　」。1　「　：　」「　：〇〇」「　：…」と、
が　でい　す。　さ　こ　です。

1　（　・　・　かどうか）　、1つ
と　す。

```
// { }　　※
const employee = {
  //　　:
  employeeNo: 123,
  name: "　　",
  isAdmin: false,
};

// "　　"
console.log(employee.name);
// "　　"　　き
console.log(employee["name"]);
// 123
console.log(employee.employeeNo);
```

す　と：

- `{　:　,　:　}` …　「ー（　）:　」　で　。
- `　.　` …　で　（　）。
- `　["　"]` …　で　（ーが　とき　ど　う）。

`employeeNo`、`name` ど。

「 」と す。

ー ーと す ー 、 す て

です。

- `Login.tsx`：`api.post("/api/login/", { employee_no: Number(employee_no) })` → `{ employee_no: 123 }` という ー ー （§18）。
- `KosuList.tsx`： ー `item` `{ id, employee_no3, name, work_day2, tyoku2, judgement }` という 。`item.id`、`item.work_day2` う で し す（§19）。

が、さ ってい こと あ す。

```
//  ※
const response = {
  status: 200,
  // data が、
  data: {
    message: "  ",
    results: [1, 2, 3],
  },
};

// "  "  げ く
console.log(response.data.message);
// 1  → →
console.log(response.data.results[0]);
```

axios `response.data.results`、`err.response?.data.message` う 、 でつ いで し す（§18・§19）。「 」 ってい です。

ここ JavaScript 3 。 で ていす。

か 、 して すき です。

> か `name` と `age` 、そ ぞ
> し す。

```
//  ※
const employee = { name: "    ", age: 42, isAdmin: false };

//      き
const name1 = employee.name;
const age1 = employee.age;

//                  す
// employee.name → name、employee.age → age
const { name, age } = employee;
// "    "
console.log(name);
// 42
console.log(age);
```

- `const {  ,  } =  ;` … じ 、 す が 。
  で え す（こち で す）：

```
// first="a", second="b"  ※
const [first, second] = ["a", "b", "c"];
```

> `Login.tsx` `const [employee_no, setNumber] = useState<string>("")`（§18） 、 です。`useState` が す 0 `employee_no` 、1

> `setNumber` という　で　け　ってい　す。React　`useState`　こ　で　ず　い　す（`05`　で　）。

`...`（　　3つ）で、　　し　す。

> 　　しい　　ーしつつ、
> 　し　す。

```
//　※
const base = { page: 1, size: 20 };
// base　　し、filter
const extended = { ...base, filter: true };
// extended　{ page: 1, size: 20, filter: true }

const arr1 = [1, 2];
// arr1　　し　[1, 2, 3, 4]
const arr2 = [...arr1, 3, 4];
```

- `{ ...　　, 　　: 　}`…　　げて、　しい　　す。
- 　　さず「　ー＋　」でき　で、React　　で　し　す（`05`　）。

> 　　`KosuList.tsx`　`fetchData`　：

```
params: {
  page: page,
  ...(day && {　　// day　があ　き　け、こ　　し　す
    day: day,
    mode: mode ? "month" : "day",
    filter: "true",
  })
}
```

こ 「



JavaScript 「 て いい」 です。で そ さが、 す。

い だ け してください。「 」と って した
だった——こういう が き す。 、 「こ 」「こ 」と
こと。 った うとす と、 してく
す。

```
// JavaScript し ※
let age = 42;
// い す か い
age = " ";
// " " * 2 → NaN じ い いう
const result = age * 2;
```

```
// TypeScript あ ※
// age 、
let age: number = 42;
// ❌ い が
age = " ";
```

が
じったとき ど す。 た 「 え」 い す。

あと : いて、 し す。

そ ぞ 「こ しか い」と
す。

```
// string                ※
const name: string = "    ";
// number
const age: number = 42;
// boolean
const isAdmin: boolean = false;
```

- `    :    =   ;` … `:`   く。
- 　　：`string`（　　）、`number`（　）、`boolean`（　　）。

　、　　があ　　できま す。TypeScriptが　か　　してく　　か
です。

```
//　　し　、TypeScriptが「こ　　string」　　※
const name = "    ";
// ❌ 　　　　string　　さ　い　　い
name = 42;
```

> 　　　　　　　　せ、　　・　　`useState`　ど「　か
> くい　」だけ　　す　、という　　です。`KosuList.tsx`　`const days = ["  ", ...]`　　があ　せ　（　　だと　さ　）。　、　`formatTyoku = (value: string | number): string => ...`　　と　　してい　す
> （§19）。

　で　「　　」と「　　」　　できま す。

> 　　　　「　　2つ　け　、　　す」　だと　　し　す。

```
//     a,b  number、          number  ※
const add = (a: number, b: number): number => {
  return a + b;
};
```

- `(a: number, b: number)` …   `:`   。
- `): number =>` …   じ `)`   `: number` が   。

                `formatTyoku = (value: string | number): string => { ... }` （§19）。「`value`   た   、   」と   してい す。`getDayOfWeek = (dateStr: string): string => { ... }`   です。

`|`（   ）で「A た B」という   す。

```
//                  ※
let value: string | number;
// ✅ OK
value = "abc";
// ✅ OK
value = 123;
// ❌        boolean     し  い  い
value = true;
```


`interface`（　　ー　　ー　）　、　　す　「　　」
です。　　で　　TS　　です。

> 　　「　　、　（　）・　（　　）・　　か（　）　つ」という　　し、そ　　った　　す。

```
// Employee　いう　　※
interface Employee {
  //
  employeeNo: number;
  //
  name: string;
  //　　か
  isAdmin: boolean;
}

// こ
const e: Employee = {
  employeeNo: 123,
  name: "　　",
  isAdmin: false,
};

// ❌　　name が　け　い　　が　う
const bad: Employee = {
  //　employeeNo　number　ずが
  employeeNo: "123",
};
```

　す　と：

- `interface　　{　　:　　; ... }` …　　。　　`;`。
- こ　`: Employee`　　う　　で　うと、「こ　　ずこ　」と　さ　。
- 　が　う（　ーが　　い・　が　う）と、　　で　づけ　。

> 　　ー　ーか　　ー　「　」　　しておけ　、`item.name` と　くとき　　が　　してく　（　）、`item.nmae`（　　）

ー　　す。「 ー　　た・ ち　 えた」という　　させ す。

> 　　`KosuList.tsx`　　あ `interface Kosu { ... }`（§19で ）が、 さ こ です。 ー ーか 1　　ー　　してい す。

`type` で　　け　　す。`interface` と てい すが、union　ど　　け が い す。

```
// 「　　か　 」　ID　いう　　け　※
type ID = string | number;
//　　け　 す
type Status = "success" | "error" | "loading";
```

> 　　「　　」 `interface`、「union 」 `type`、と えておけ　　で　　です。
> `interface`（`Kosu`）　 ってい す。

こ で　　ー です。　　「 ー ーと す 」で、たとえ でじっく　　し し う。

> - … 1つ　　が　　で、　　ず　 こと。
> - …　　かか　　こと。

あ たが し した。 ができ で かか す。

- した か あ た し、 が で10 、 ず せ ず 。 (お さ い)。
- した 、あ た で した だ でき 。 ができた が でく 。 。

Web で「 ー ー ー い せ 」 、 さ 「 す 」 う 。 し がかか す。そ 、 す が です。 し で いた 、 ー ー つ 、 が できく ってし い す。

す が です。

「 ・ し 」 し す。Promise 3 つ ち す:

- … (って)。
- … (ができた)。
- … (で かった)。

(axios、§17) 、`api.post(...)` と 。 ー だ てい せ 。

Promise た が `async` と `await` です。

ー ー 、そ 「 が で ってか 」 す。こ `Login.tsx` そ です。

```
// ① async  こ                          ※
const handleSubmit = async (event) => {
  // ② await      が
  const response = await api.post("/api/login/", { ... });
  // ③     が    か  、こ    が    さ
  const data = response.data;
  console.log(data);
};
```

す と：

- … け 。「こ で `await` う（ ）」という 。
- … Promise（ ） け 。「こ 」という 。
- `await` け と、`api.post` が したPromiseが （ が ）す で、そ で し す。 （ だか ）。 が た 、 き す。

> `await` `async` けた でしか え せ 。「`async` で『こ です』と し、そ で `await` で『こ つ』」という です。

> `await` が いと、Promise `.then(...)` という ー （ ）で け があ 、 が って くく す。`await` うと、 ー す。こ が `async` / `await` です。

（ ー ー 、 、 ー い ど)。 え が `try` / `catch` です。

> (try)、 し した (catch) ー し す。

```
// try  こ            「  す」  ※
try {
  //      す
  const response = await api.post(...);
  console.log("    ", response);
// catch  try            が  き    、ここ        く
} catch (err) {
  //   err            が      い
  console.log("    ", err);
// finally                          ず
} finally {
  console.log("    お    ");
}
```

す と：

- `try { ... }` …「 す か し い 」 。
- `catch (err) { ... }` … `try` で ー（ ）が き と、 ここ 。`err` ー が 。
- `finally { ... }` … ・ （ け 。 ： ー す）。

> `try` 「 」、`catch` 「 った 」。 ちて （ ー が きて ） が け 、 が す ぎ す。

> - `Login.tsx`：`try` で 、`catch` で ー ー （§18）。
> - `KosuList.tsx`：`try` で ー 、`catch` で ー 、`finally` で ー （§19）。 `try/catch` で 、と えてください。

、 ー し す。 で ー け す が `import` / `export` です。

1つ で った 「 」し、 で「け って う」 し す。

```
//  A math.ts ※
// export こ add か え う す
export const add = (a, b) => a + b;
```

```
//  B ※
// math.ts か add
import { add } from "./math";
// 5
console.log(add(2, 3));
```

- `export` …「こ す 」 。
- `import {  } from "  "` … か 。`./` 「 じ 」 す 。

き	`export const add = ...`	`import { add } from "..."`	`{ }` で 。
	`export default Login;`	`import Login from "..."`	`{ }` し。1 1つ

- `axios.ts`：`export default api;` → `Login.tsx` で `import api from "../api/axios";`（ ）。
- `Login.tsx`：`export default Login;` → で `import Login from ...`（ ）。

- `KosuList.tsx`：`import React, { useState, useEffect, ... } from "react";` → React（　　）と `useState`　（　　き）　　。
- `import { Link, useNavigate, useLocation } from "react-router-dom";` →　　きで　　。

　　import　　あ　　です。

- `./` …　　と　　。
- `../` …　　。`../api/axios`　「1つ　　ってか　api axios」。
- `"react"`　う　`./`　`../` が　か　い　　、（`node_modules`）か　　す。

ここか　　ー　です。　ず　　い `frontend/src/api/axios.ts` す。こ　　、　さくて　　です。

`api.get(...)`（　）　`api.post(...)`（　）で、　ー　ーと　ー　　し　す。§15で　だ　、こ　Promise　す　　です。

「　ー　ー　　」「　　（ー）　　」「　　（CSRF　ー　）　」　あ　かじ　　だ　　1つ　、　　で　い　せ　う　　し　す。

```
// ① axios
import axios from "axios";
// ②
const api = axios.create({
    // ③
    baseURL: process.env.REACT_APP_API_BASE_URL,
    //
    withCredentials: true,
    //   CSRF            が
    xsrfCookieName: "csrftoken",
    //
    xsrfHeaderName: "X-CSRFToken",
//                          じ
});
//          api
export default api;
```

1　ずつ　しく：

　　axios　　す（§16）。from "axios"　./　が　いてい　い　で、こ　　ー　　（node_modules）か　です。こ　で axios という　が　え　う　　す。

　　axios.create(...) で、　、api という const　　す（§2）。　　{ ... }　（§12）で、　～　　が　ー:　　で　でい　す。

> 　　axios.get("http://　　/...", {　　})　と　くと、　ー　ー　　こと　、　　とです。
> 1　api　　っておけ　、　　で　api.get("/api/...") と　　さ　す。「　　した　1　　す　」　ー　です。

　　ー　ー　　し　す。

- process.env.REACT_APP_API_BASE_URL … .env　（03　　で　た　）　か　た　　。　え　https://　、http://localhost:8000　う　、　　です。

「　　」と「　　」で　ー　ー　　えたいとき　い　す。
ー　　き　えず　　え　　が　です。`process.env.〇〇` で
す。React で　　`REACT_APP_` で　　ー　があ　す。

　　こ　　しておくと、　　で `api.post("/api/login/")` と　くだけ
で、　　`baseURL + "/api/login/"`（　：`http://localhost:8000/api/login/`）という
　　してく　す。`Login.tsx`　`api.post("/api/login/", ...)` が　く
け　　こ　おかげです。

「　　とき　　」という　です。

で　「　　かどうか」　す　　がここ　す。`withCredentials: true`　し　いと、せっかく　　して　　ーが　　ず「　だか　か　　い」　い
す。CLAUDE.md　あ　「　　」　させ　た　　です。

　　です。

　　こ　　ぐた　、　ー　ー　「　　（CSRF　ー　）」
し、　ー　　そ　　せ　す。

- `xsrfCookieName: "csrftoken"` …　ー　ーが　　てく　　。
- `xsrfHeaderName: "X-CSRFToken"` … そ　　。こ　2
で「　　ーか　　し、　　った　　ーで　　す」
axios が　　で　ってく　す。Django（　　）　　CSRF　　せた
です。

（
ー　）と　　、「こ　　か　か」「　　こ　」　ど　　す。

った `api` （§16.2）で し す。こ 、 が `import api from "../api/axios";`（`Login.tsx` 3 、`KosuList.tsx` 2 ）で け って え す。

> こ 8 「 で 、 `api` 1 って 」 。 す て 、こ `api` import して `api.get` / `api.post` で ー ーと し す。
> です。

、 `handleSubmit` 1 ずつ す。「 した が き か」 です。`03` で こ JSX（ た ） したが、 そ す。

ず、 す ー か 。

> 、「 さ た 」「 ー ー 」という2つ と、 ー し す。

```
// ① React       useState
import React, { useState } from "react";
// ②
import { Link, useNavigate } from "react-router-dom";
// ③ §17
import api from "../api/axios";
//                 う axios
import axios from "axios";
//
import logo from "../img/TitleRogo.png";
//   こ          CSS  03
import styles from "../styles/MainPage/Login.module.css";

//   Login
const Login: React.FC = () => {
  //
  const [employee_no, setNumber] = useState<string>("");
  //
  const [errorMessage, setErrorMessage] = useState<string>("");
  //
  const navigate = useNavigate();
```

- … import（§16）。React 、 ー （ ） 、§17 `api`、ー `axios` 、 、CSS でい す。`react` と `react-router-dom` 、`../` き です。
- … `Login` という （ 。`05` で ） ー で 。`: React.FC` 「こ React ー という です」という （§14）です。
- … ここが 。
  - `useState`（`05` で ）「 と す 」 React 。
  - `<string>` （§14）で「こ 」 。
  - `("")` 「 」。
  - で、そ （§13.1）して、`employee_no`（ ）と `setNumber`（ え ） け ってい す。
  - つ 「 さ た えておく と、そ す 」 。
- … 、 ー ー `errorMessage` と `setErrorMessage`。（= ー し）。
- … 「 ー す 」 `navigate` 。で `navigate("/")` う い、 し す。

こ が したとき です。§15で だ `async` / `await` / `try` / `catch` が し す。

> ー 、 ー し、 ー ー って 、 ー 、 し す。 で `async/await`、 えて `try/catch` い す。

```
// ①             し
const handleSubmit = async (event: React.FormEvent) => {
  // ②
  event.preventDefault();
  // ③                    す
  setErrorMessage("");

  //          す          え
  try {
    //                 し
    const response = await api.post("/api/login/", { employee_no: Number(employee_no)
});
    //                    す
    const data = response.data;
    //          い  あ
    if (data.status === "success") {
      //
      navigate("/");
    //    う    け
    } else {
      //
      setErrorMessage(data.message || "             が    し  し  。");
    }
  //          が    し
  } catch (err) {
    //   axios
    if (axios.isAxiosError(err)) {
      //   401(      )
      if (err.response?.status === 401) navigate("/login");
      //
      else setErrorMessage(err.response?.data.message);
    //   axios
    } else setErrorMessage("          が    し  し  。IT          し  く  さ
い。");
  }
};
```

1　ずつ　しく：

　　`handleSubmit`（§9）で　。　`async`（§15.3）で「　で`await`　う　」だと　。`event`　「　ー　（ど　が　きたか　）」で、`React.FormEvent`（§14　）。

「　　が　さ　た」「　　ー　が　さ　た」「　ーが　さ　た」　ど。そ　　が event　　って　　さ す。

　　preventDefault()　「　　」　。

　　HTML　<form>　、　　すと　　して　ー　ー　　うとし　す（　が　　き）。そ　こ　1　で　、　　たち　handleSubmit（JavaScriptで　　）で　し　す。こ　が　いと、　した　　ー　が　ー　さ　てReact　　が　えてし　い　す。　　です。

　　ー　　ー　　し、　　（§18.1　setErrorMessage）。　　た　　い　　ーが　　い　う　す　　です。

　　ここか　「　す　か　し　　い　　」　　（§15.4）。　で　　ーが　きた　catch　　す。

★　　★こ　1　が「　ー　ー　　す　」　です。　し　す：

- api.post(...) … §17で　った api　　い、　ー　ー　　（　ー　　）　す　。Promise（§15.2、　　）　す。
- "/api/login/" …　　。baseURL と　さって　URL　　（§17　）。
- { employee_no: Number(employee_no) } …　　ー　（　　、§12）。　ー　employee_no　、　　した　　。
- Number(employee_no) … ★　　★　か　　（03 §2.2　、<input type="number">　で　　で　く）。Number(...) で　　してか　ってい　す。　ー　ー　　していた　です（　　え　ぐ）。
- await … こ　　（§15.3）。　ってい　、　　せ　。
- const response = ... …　ってきた　（　ー　ー　）　response　け　。

- 　…　ー　　す　（　）。KosuList　　ど。

> - … ー （ く）。 ど。 「 って して う」 で POST です。

ー ーか response data （ ） し す（§12.1 ）。response ー ど々 ってい すが、しい data です。こ data 、 ー ーが した { status: ..., message: ... } う が ってい す。

data status が "success" と（§4.3 ===） 。 ー ーが「 」 status: "success" で せてく ってい です。

、navigate（§18.1 ）で 。 す。

status が "success" で かった （= ー ー したが「 」と ってい 、 。 : が さ てい い)。

サーバーエラーが発生しました。

す た 、 ー ー 。

- data.message … ー ーが した （ :「 す がい せ 」)。
- || " が し し 。" … §4.4 「 いとき 」。 し data.message が 、 「 ー ー ーが し した。」 。
- ここで errorMessage が と、JSX（03 §3 {errorMessage && ...}）が し て い ー が す。

ここか try で が きたとき（§15.4)。 、 ー ー 、 （401） ど ここ す。err ー が す。

axios.isAxiosError(err) 「こ ー 」 す （ でimportした axios)。 ー 、err.response（ ー ー ）か しい が す。

err.response?.status （§13.3 ?. で ）が 、 。

> `200` = 、`401` = （ が ）、`403` = し、`404` = つか い、`500` = ー ー ー。`401` 「あ たが だか か い（ して）」 で、 す。CLAUDE.md 「401 」という が、こ で さ て い す。ここで `?.` う 、 ー どで `err.response` が し い ことがあ 、そ が し い う す た です。

401 ー 、 ー ーが した ー ー（`err.response?.data.message`） し す。

不明なエラーが発生しました。 担当者に連絡してください。

`axios.isAxiosError(err)` が false = axios ー。 で、「IT 」 す ー し す。

> - しい て → ー （ ）。
> - し い → い ー で が さ （ ）。
> - ー ー た で →「 ー…」 た ー ー （ ）。

> 「 → い ー し→ し → 、 → ～ ー 」。
> です。 、 こ していい す。

、 `KosuList.tsx` か 、こ ー す ——**`interface Kosu`**・**`formatTyoku`**・**`getDayOfWeek`**・ ー ・**`map`**—— 1 ずつ す。

ー　ーか　「　1　」　ー　が、ど　ーとど
つか　す　です（§14.6）。

```
// 「　1　」　す
interface Kosu {
  //　。　す
  id: number;
  //
  employee_no3: number;
  //
  name: string;
  //　。"2026-06-01"　う
  work_day2: string;
  //　。　。"1"〜"6"
  tyoku2: string;
  //　。true　OK、false　NG
  judgement: boolean;
}
```

1　ずつ：

- `interface Kosu {` … `Kosu` という　（§14.6）。
- `id: number;` …　ー `id`　。　ー　す　。　で `key={item.id}` URL　う。
- `employee_no3: number;` …　、　。
- `name: string;` …　、　。
- `work_day2: string;` …　。　ですが、　ー　ーか　`"2026-06-01"`　う　で　で `string`。
- `tyoku2: string;` …　（　）。`"1"`〜`"6"`　。こ　`formatTyoku` で「1　」　ど　し　す。
- `judgement: boolean;` …　。`true`（OK）か `false`（NG）（§3.1）。
- **　`;`**。

で `useState<Kosu[]>([])` と くことで、「`data` `Kosu` だ」とTypeScript す。す と `item.work_day2` と け が してく 、`item.workday2`（ ） ー す。 おかげで、 です。

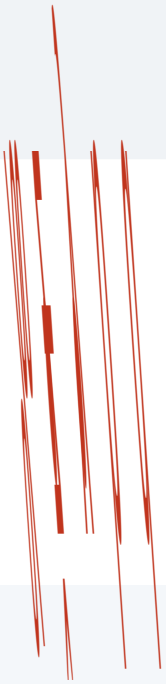
`Kosu[]` 「Kosu がたくさ だ 」。`number[]` 、`string[]` 。§14 です。

（1〜6、 で で ） け 、「1 」「 」 ど して し す。§6.3 switch、§14.4 です。

```
//       string|number、         string
const formatTyoku = (value: string | number): string => {
  // value         し
  switch (Number(value)) {
    // 1      "1  "     す
    case 1: return "1  ";
    // 2      "2  "
    case 2: return "2  ";
    // 3      "3  "
    case 3: return "3  ";
    // 4      "    "
    case 4: return "    ";
    // 5      "  1  "
    case 5: return "  1  ";
    // 6      "  2  "
    case 6: return "  2  ";
    //           け
    default: return "";
  }
};
```

1 ずつ：

- `| number): string => {`
  `string | number` (union 、§14.5) =「
  `ace` `tyoku2` ですが、 た で
  （§14.4）=「 ず す」。
- してか （§18 と じ `Number`)。
  で、 してか `case 1` どと し す。
- して ）。



>

>



1　ずつ：

- `const getDayOfWeek = (dateStr: string): string => {`
  - 　　ー　　。　`dateStr`　　（　　）、　　（　）。
- `const days = ["　", "　", "　", "　", "　", "　", "　"];`
  - 　　す7　　（§10.1）。　　：`days[0]`が「　」、`days[1]`が「　」…`days[6]`が「　」。こ　　`getDay()`　　（0=　　〜6=　）　　させてあ　す。
- `const date = new Date(dateStr);`
  - `new Date(...)`で、　　か　　す。

    > 　　`new Date("2026-06-01")`　　う　　すと、そ　　す　　ができ、`.getDay()`（　）　`.getMonth()`（ ）　どで　　せ　す。`new`「　しく　」　ー　ー　（`05`　で　）。
- `return days[date.getDay()] || "";`
  - `date.getDay()`… そ　　す（　=0、　=1、…、　=6）。
  - `days[ ... ]`… そ　　で　　く。　え　2026-06-01が　`getDay()`　1、`days[1]`　「　」。
  - `|| ""`…「　いとき　」（§4.4）。　`dateStr`が　で`getDay()`が`NaN`　すと`days[NaN]`　`undefined`　た　、そ　　して　ー　ぎ　す。

> 　　どち　「　ー　ーか　た　ー　（　・　）　、　す　」　。こういう　　と　す。　　で　い　す。

ここで§13.2（　　）・§4.4（`||`）・§6.2（　）・§15（async/try/catch）が　し　す。

> 　　ー　ー　「　　ください。　ー　　こ　、（　　があ　）こ　で　　で」と　　で　い　せ、　　し　す。

ー　　し、　　ず　ー　　し　す。

```
const fetchData = useCallback(async (
  //
  page: number,
  //           す                          し
  day: string,
  //                 true、           false
  mode: boolean
) => {
  // ①
  setLoading(true);
  // ②         す
  try {
    // ③       GET
    const response = await api.get("/api/kosu_list/", {
      //               す
      params: {
        //                  ず
        page: page,
        //   day     があ    き  け、          し
        ...(day && {
          //
          day: day,
          //       か      か
          mode: mode ? "month" : "day",
          //
          filter: "true",
        })
      }
    });

    //               け
    const results = response.data.results || [];
    //   1             け    20
    const pageSize = response.data.page_size || 20;
    //
    setData(results);
    //                   し
    setTotalPages(Math.ceil(response.data.count / pageSize));
  //
  } catch (err) {
    //   axios
    if (axios.isAxiosError(err)) {
      //   401
      if (err.response?.status === 401) navigate("/login");
      //   403        し
      else if (err.response?.status === 403) navigate("/");
      //
      else setError(err.response?.data.message);
```

```
    } else {
      //   axios
      setError("          が   し  し  。IT          し  く  さい。");
    }
  //
  } finally {
    //
    setLoading(false);
  }
}, [navigate]);
```

  って し す（useCallback 05 Reactで 。ここで し す）：

- [illegible] … 。こ で 「 …」（Loading）が す（104 で ）。
- [illegible] … 。Login.tsx 、 ず try で す。
- [illegible] … api.get（§17 api）で ー （GET）。await で つ。 で post でしたが、こち で get です。
- [illegible] … GET （ ー ）。{ page: 2 } 、URLが /api/kosu_list/?page=2 う 、 ー ー が す。
- [illegible] ★ ★ … §13.2 と§4.4 && せ 。
  - day && { ... } … day（ ） があ { day, mode, filter } 、 ""（ さ い ） す。
  - ... で、そ params す 。
  - :「 page だけ ／ page + day + mode + filter 」という、 い て ってい す。
  - mode: mode ? "month" : "day" … （§6.2）。mode が true（ ） "month"、false（ ） "day"。
- [illegible] … results（ ー ） す。|| [] で「 け 」（§4.4）。こ で .map が ず して き、 ー せ 。
- [illegible] … 1 ー あた 。 け 20。
- [illegible] … した data 。こ で が す。

- [illegible] … ー して 。
  - `response.data.count` … ー （ ：45 ）。
  - `count / pageSize` … 45 / 20 = 2.25。
  - `Math.ceil(...)` … 。2.25 → 3。45 20 ずつ す と3 ー 、という 。

    > [illegible] `Math.ceil(2.25)` 3。「 があ ー が 」 で げが です。`Math` と た （`Math.floor` て、`Math.round` ）。
- … `Login.tsx` と じ ー 。 い で とき 。「 してい が、こ が い」 です。
- [illegible] … で で ー す（§15.4）。`finally` が け 、 ー 「 …」が っ し ってし い す。

> `try` = 、`catch` = （ ）、`finally` = い け（ ー ）。こ 3 が です。

、 した ー す です。§11 `map`、§6.2 、§5 ー がす て し す。

> ー が0 「No data found.」 し、あ ー 1 （`<tr>`） して す。 で え、 し、OK/NGで け し す。

```
// ①           が0    …
{data.length === 0 ? (
  //     「         し」
  <p>No data found.</p>
// ②  う     け     …
) : (
  <TableContainer ...>
    <table>
      <thead> ... </thead>
      <tbody>
        // ③ data              item    、
        {data.map((item) => (
          //    1   。key             id
          <tr key={item.id}>
            //
            <td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>
            //
            <td>{formatTyoku(item.tyoku2)}</td>
            //    OK/NG       け
            <td className={item.judgement ? styles["status-ok"] : styles["status-ng"]}>
              //
              {item.judgement ? "OK" : "NG"}
            </td>
            <td>
              //
              <Link to={`/kosu-update/${item.id}`} className={styles["a-collar"]}>
</Link>
            </td>
            <td>
              //
              <Link to={`/kosu-delete/${item.id}`} className={styles["a-collar"]}>
</Link>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
    <Pagination ... />
  </TableContainer>
)}
```

　　：

- 　　…`data.length`（　　、§10.1）が0か`===`で　し、　　（§6.2）で「0　　、あ　」　し　け。こ　「って　　え　」React　　（`05`　で　）。

- ★こ ★… `data`（`Kosu[]` ） （§11.1 map)。「 ー が10 あ 10 `<tr>` ができ、 10 さ 」。Reactで す ー です。
- … `key`（ ）として `item.id` 。

> React 、 ー が したとき「ど がど か」 `key` で け て く し す。`map` で とき け が ー です（`05`)。`interface Kosu` で `id` たせていた 、 さ こ た で あ す。

- … `item.work_day2`（ ：`"2026-06-01"`） し、そ `getDayOfWeek`（§19.3）で た きで え （ ：`2026-06-01 ( )`)。`item.work_day2` §12 、`(...)` です。
- … `item.tyoku2`（ ：`"1"`） `formatTyoku`（§19.2）で「1 」 して 。
- … で 。`judgement` が true `status-ok`（ ど)、false `status-ng`（ ど） て 。`styles[...]` CSS Modules（`03` §13)。
- … `judgement` 、 け 「OK」「NG」で （ 、§6.2)。
- … （§5）で、`item.id`（ ：5） で `/kosu-update/5` という URL 。 「 」「 」が、そ ー ID した ー う す。

> 、 （ き)・ (1 ど)・ (OK /NG)・ / が だ が 。 ー が け 「No data found.」。

interface（§14.6）	`interface Kosu { id: number; ... }`

（§14）	`useState<Kosu[]>([])`
union （§14.5）	`formatTyoku(value: string | number)`
（§14.4）	`(dateStr: string): string =>`
— （§9）	`const formatTyoku = (value) => {...}`
switch（§6.3）	`formatTyoku` `switch(Number(value))`
と （§10）	`days[date.getDay()]`
async/await（§15）	`await api.get(...)`
try/catch/finally（§15.4）	`fetchData` —
（§13.3）	`err.response?.status`
+&&（§13.2/§4.4）	`...(day && { ... })`
`||` （§4.4）	`response.data.results || []`
（§6.2）	`item.judgement ? "OK" : "NG"`
map（§11.1）	`data.map((item) => <tr>...)`
— （§5）	`` `/kosu-update/${item.id}` ``
`===` （§4.3）	`data.length === 0`

`Cannot read properties of undefined` で が っ	`null` / `undefined` `.` で った	`?.` う （§13.3）。Consoleで が undefined か
がう くいか い	`=` （ ）と `===` （ ） え	ず `===` 。`if (x = 5)` 、`if (x === 5)` が しい（§4.3）
`"5" === 5` が false で	と	`Number(...)` `String(...)` で え （§18 ）
が `NaN`	が	`Number()` で （§14.1・§18 ）

`await` けた ー	`async` が いてい い	`await` `async` だけ （§15.3）
ー が	`response` と	

こ でJavaScript／TypeScript が、 ー と つき した。 、こ って 、React す。`useState` `map` がどう く か、い い が え す。

、「05_React_ .md」 す。